

TXFETCH

Полная техническая документация

*Архитектура · Code Walkthrough · Алгоритм scoring · Разбор по сетям
by xaynov (@ownstates)*

1. Обзор

TXFetch — утилита командной строки для поиска транзакций в блокчейне по косвенным данным. Она решает обратную задачу: вместо того чтобы знать хеш транзакции и найти её данные, пользователь знает данные и хочет найти хеш.

Типичный сценарий: человек знает что отправил или получил перевод примерно в определённое время и на определённую сумму, но хеш транзакции утерян или неизвестен. TXFetch запрашивает данные у публичных API блокчейнов, фильтрует результаты и возвращает список кандидатов с оценкой вероятности совпадения.

Сеть	Монеты	API
TRON	USDT, USDC	TronGrid Events API
TON	TON, USDT, NOT	TONAPI / TONCenter v3
ETH	USDT, USDC, ETH	Etherscan
BSC	USDT, USDC, BNB	BscScan
Base	USDC, ETH	Basescan
Solana	SOL, USDC, USDT, RAY, JUP	Public RPC / Helius
Bitcoin	BTC	mempool.space
Litecoin	LTC	litecoinspace.org
Dogecoin	DOGE	dogechain
Bitcoin Cash	BCH	blockchair

2. Архитектура

Проект построен на трёх паттернах: единая точка входа, адаптер-паттерн для сетей и общий модуль scoring.

```
txfetch/  
├─ main.py          # точка входа — TUI, argparse, pipeline поиска  
├─ pyproject.toml   # конфиг пакета и CLI команды  
├─ requirements.txt  
├─ .env.example  
├─ modules/  
│   ├── models.py   # TXQuery, TXCandidate, score_candidate,  
│   │               amount_matches  
│   ├── config.py    # API ключи и константы из .env  
│   ├── matcher.py   # универсальный фильтр и scorer  
│   ├── tron.py      # TRON адаптер  
│   ├── ton.py       # TON адаптер  
│   └─ evm_base.py   # базовый класс EVM сетей
```

```
|— eth.py          # Ethereum
|— bsc.py          # BNB Smart Chain
|— base_chain.py   # Base
|— solana.py       # Solana адаптер
|— utxo_base.py    # базовый класс UTXO сетей
|— btc.py          # Bitcoin
|— ltc.py          # Litecoin
|— doge.py         # Dogecoin
|— bch.py         # Bitcoin Cash
```

2.1 Поток данных

Пользователь вводит параметры → `TXQuery` → адаптер запрашивает API → `matcher.find_matches()` → список `TXCandidate` отсортированный по `confidence`.

2.2 Паттерны

Adapter pattern

Каждый адаптер реализует один метод — `fetch_candidates(query) → list[TXCandidate]`. `main.py` не знает о внутренних различиях между сетями. Добавление новой сети = новый файл + одна строка в словаре адаптеров.

Base class pattern

`EVMBase` содержит всю логику для Ethereum-совместимых сетей: конвертация `timestamp` в блок, запросы `tokenTx`, пагинация. `ETH`, `BSC`, `Base` наследуют его и добавляют только конфиг (`URL`, токены, ключ).

`UTXOBase` содержит логику для Bitcoin-подобных сетей: оценка высоты блока, пагинация блоков, парсинг `vout/vin/OP_RETURN`. `BTC`, `LTC`, `DOGE`, `BCH` наследуют его.

Dispatch table

Вместо длинного `if/elif` — словарь адаптеров. `importlib` динамически импортирует нужный модуль по ключу сети. Добавление сети не требует правки логики `run_search`.

3. Code Walkthrough

Разбор каждого файла простым языком — что делает, зачем существует, как связан с остальными.

main.py — точка входа

Это первый файл который запускается. Он отвечает за три вещи: показать интерфейс, собрать параметры от пользователя, запустить нужный адаптер и вывести результат.

Внутри два режима работы. Если пользователь запустил просто `txf` — открывается TUI (текстовый интерфейс) на `prompt_toolkit`: ASCII-арт, меню выбора сети стрелками, пошаговый ввод параметров. Если передал флаги (`txf -n tron -c USDT ...`) — `argparse` собирает их и сразу запускает поиск без интерфейса.

Ключевые функции:

- `select_network()` — меню выбора сети стрелками через `prompt_toolkit.Application`
- `ask_params()` — пошаговый ввод: дата, время, выбор окна, монета, сумма, мемо
- `run_search()` — собирает `TXQuery`, вызывает нужный адаптер через словарь, обрабатывает мемо fallback
- `_print_results()` — выводит таблицу, предлагает скопировать TXID или сохранить в Excel

и мемо fallback: если указан мемо но ничего не найдено — автоматически повторяет поиск без него и предупреждает пользователя.

models.py — общие типы данных

Содержит три сущности которые используются везде: `TXQuery`, `TXCandidate` и функции `scoring`.

`TXQuery` — это объект запроса. `main.py` собирает его из ввода пользователя и передаёт в адаптер. Содержит: временное окно (`time_from` / `time_to` в UTC), монету, опциональную сумму (или диапазон `amount..amount_to`), `tolerance` и мемо.

`TXCandidate` — результат поиска. Все адаптеры возвращают список таких объектов, поэтому `main.py` работает одинаково для любой сети. Содержит: хеш транзакции, время, сумму, адреса, мемо и `confidence`.

`amount_matches()` — отдельная функция проверки суммы. Поддерживает два режима: точное совпадение с `tolerance` и диапазон (если задан `amount_to`). Используется внутри `score_candidate`.

`score_candidate()` — вычисляет `confidence` по трём критериям: время, сумма, мемо. Если сумма задана но не совпадает — немедленно возвращает 0.0 без дальнейших вычислений.

config.py — конфигурация

Читает файл `.env` из корня проекта при старте и отдаёт значения через именованные константы. Никаких внешних зависимостей — чистый Python.

Две категории значений: API ключи (`TRONGRID_API_KEY`, `ETHERSCAN_API_KEY` и т.д.) и runtime константы (`REQUEST_TIMEOUT`, `PAGE_LIMIT`, `MAX_PAGES`, `PAGE_DELAY`, `MIN_CONFIDENCE`). Все имеют дефолтные значения — если ключ не

задан, адаптер работает в публичном режиме. Если константа не задана — используется дефолт.

и config также смотрит в `os.environ` — можно передавать ключи через переменные окружения без файла `.env`.

matcher.py — фильтрация и scoring

Единственная публичная функция: `find_matches(events, query, parser, network, min_confidence)`.

Принимает сырые события от API, функцию-парсер (как достать поля из конкретного формата), `query` и название сети. Внутри: для каждого события вызывает `parser` → получает (`timestamp`, `amount`, `memo`, `from`, `to`, `txid`) → вызывает `score_candidate` → отбрасывает ниже порога → сортирует по убыванию.

Зачем отдельный модуль: раньше каждый адаптер копирастил один и тот же цикл фильтрации. После рефакторинга логика оценки живёт в одном месте, адаптер только определяет парсер — как достать поля из своего формата API.

tron.py — TRON адаптер

Самый простой адаптер. TronGrid поддерживает `min/max_block_timestamp` в запросе событий контракта — сервер сам обрезает выборку по времени. Адаптер только пагинирует через `fingerprint cursor` и передаёт события в `matcher`.

Особенность: при ошибке запроса делает до 3 повторных попыток с паузой перед тем как остановиться. Предупреждает пользователя если результаты могут быть неполными.

ton.py — TON адаптер

Работает в двух режимах в зависимости от монеты. Для Jetton (USDT, NOT) — `TONAPI /jettons/{id}/transfers` с параметрами `start_date/end_date`. Для нативного TON — `TONCenter v3 /transactions` с `start_utime/end_utime`.

TON — единственная сеть где `memo` является нативным полем протокола. Комментарий к переводу хранится в `message_content.decoded.comment` и извлекается автоматически. Это не хак — стандартная возможность блокчейна.

Пагинация через `before_lt` — `logical time` последней записи. Это не Unix timestamp, а внутренний порядковый номер TON который гарантирует уникальность позиции в хронологии.

evm_base.py + eth.py / bsc.py / base_chain.py — EVM адаптеры

EVMBase содержит всю общую логику. Конвертация времени в блоки (два вызова `getBlocknobytime`), запрос `tokenTx` по диапазону блоков, пагинация `page/offset`. Каждая конкретная сеть — это 10-15 строк конфига: URL, ключ, словарь токенов с адресами и `decimals`.

⚠ BSC USDT использует 18 decimals, а не 6 как на Ethereum. EVMBase применяет правильное значение автоматически из конфига токена.

solana.py — Solana адаптер

Самый сложный адаптер из-за особенностей архитектуры Solana. Нет глобального endpoint для фильтрации по времени — используем `mint address` токена как точку входа.

Два режима в зависимости от наличия Helius ключа. Без ключа:
`getSignaturesForAddress(mint)` → фильтруем по `blockTime` → `getTransaction` для каждой подписи → парсим `preTokenBalances/postTokenBalances`. С ключом Helius:
`/v0/transactions` принимает список подписей и возвращает уже готовые поля `tokenAmount`, `fromUser`, `toUser`, `memo` — в разы быстрее.

Мемо извлекается двойной проверкой: точный `programId` программы Мемо (`Mem_oMoDT32...`) и fuzzy проверка 'memo' в названии программы как fallback. Ищем в основных `instructions` и в `meta.innerInstructions` — последнее важно для DEX-транзакций где перевод происходит внутри другой программы.

utxo_base.py + btc.py / ltc.py / doge.py / bch.py — UTXO адаптеры

UTXOBase содержит логику для Bitcoin-подобных сетей. Ключевой параметр `BLOCK_TIME` — среднее время блока в секундах (600 для BTC, 150 для LTC, 60 для DOGE). Используется для оценки высоты блока по `timestamp`: `tip_height - (now - target_ts) / BLOCK_TIME`.

Сумма транзакции — это сумма всех non-OP_RETURN выходов (`vout`). Включает сдачу отправителя. Это неточность UTXO модели — рекомендуется использовать широкий `tolerance` при поиске BTC-транзакций.

Мемо через OP_RETURN: специальный тип выхода куда некоторые сервисы записывают произвольный текст. Декодируется из hex в UTF-8. В большинстве стандартных переводов отсутствует.

4. Алгоритм scoring

После получения транзакций от API каждому кандидату присваивается `confidence` от 0 до 1. Кандидаты ниже порога отбрасываются, остальные сортируются по убыванию.

Критерий	Вес	Логика
----------	-----	--------

Критерий	Вес	Логика
Время	0.5	Линейно от time_from. TX в 20:38:00 → 1.0, TX в 20:38:59 → ~0.0
Сумма	0.4	Точное совпадение → 1.0. В пределах tolerance → линейно. Вне — немедленный отброс.
Мемо	0.1	Case-insensitive substring match → 1.0. Нет совпадения → 0.0.

4.1 Формула

`confidence = 0.5 × time_score + 0.4 × amount_score + 0.1 × memo_score`

4.2 Диапазон суммы

Если пользователь вводит 500-800 вместо конкретного числа — активируется режим диапазона. Транзакция проходит если $\text{amount_from} \leq \text{event_amount} \leq \text{amount_to}$. Score внутри диапазона линейно убывает от центра к краям — TX ровно в середине диапазона получает 1.0.

4.3 Перераспределение весов

Если критерий не задан — его вес перераспределяется на остальные:

- Нет суммы и нет мемо → $\text{confidence} = \text{time_score}$
- Нет суммы, есть мемо → $0.9 \times \text{time} + 0.1 \times \text{мемо}$
- Есть сумма, нет мемо → веса 0.5 и 0.4 нормализуются пропорционально

4.4 Мемо fallback

Если пользователь указал мемо но поиск вернул пустой список — main.py автоматически создаёт копию запроса без мемо и повторяет поиск. Пользователь видит предупреждение. Это позволяет не терять результаты если отправитель не добавил комментарий.

5. Разбор по сетям

TRON

Параметр	Значение
Фильтр времени	min/max_block_timestamp на стороне сервера (миллисекунды)
Единицы суммы	sun: 1 USDT = 1 000 000 sun (decimals=6)
Мемо	не поддерживается в стандартном TRC-20 Transfer
Пагинация	fingerprint cursor из поля meta ответа
Ограничения	rate limit без ключа ~1 req/sec, 429 на второй странице

TON

Параметр	Значение
Jetton фильтр	start_date / end_date (Unix seconds) на стороне сервера
Нативный фильтр	start_untime / end_untime через TONCenter v3
Единицы суммы	nanoTON для TON (decimals=9), USDT decimals=6, NOT decimals=9
Мемо	нативное поле in_msg.message_content.decoded.comment
Пагинация	before_lt (logical time) для Jetton, offset для нативного TON
Ограничения	два разных API для Jetton и нативного TON

ETH / BSC / Base

Параметр	Значение
Фильтр времени	timestamp → блок через getblocknobytime, два вызова
Запрос	tokenTx?contractaddress=...&startblock=X&endblock=Y (без address)
ETH USDT decimals	6
BSC USDT decimals	18 (отличается от Ethereum!)
Мемо	не поддерживается в ERC-20 Transfer
Пагинация	page / offset, до 1000 записей на страницу
Ограничения	нужен API ключ для нормальной работы, 5 req/сек на бесплатном

Solana

Параметр	Значение
Подход	getSignaturesForAddress(mint) → фильтр по blockTime → getTransaction
С Helius	helius /v0/transactions — возвращает готовые tokenAmount, fromUser, toUser, memo
Единицы суммы	uiAmount из preTokenBalances/postTokenBalances (уже в читаемом виде)
Мемо	Мемо Program (Mem_oMoDT32...) в instructions и innerInstructions
DEX-переводы	ищем в meta.innerInstructions для транзакций через DEX
Ограничения	публичный RPC жёстко лимитирован, рекомендуется Helius ключ

Bitcoin / Litecoin / Dogecoin / Bitcoin Cash

Параметр	Значение
----------	----------

Параметр	Значение
Подход	оценка высоты блока → перебор блоков → перебор TX внутри блока
BLOCK_TIME	BTC: 600 сек, LTC: 150 сек, DOGE: 60 сек, BCH: 600 сек
Единицы суммы	satoshi: сумма всех non-OP_RETURN vout / SATOSHI
Мемо	OP_RETURN поле, hex → UTF-8. В большинстве переводов отсутствует.
Пагинация	25 TX за вызов через start_index
Ограничения	нет серверного фильтра ни по времени ни по сумме — весь перебор локально

⚠ Сумма Bitcoin включает сдачу отправителя. Используйте широкий tolerance (0.1 и более) при поиске BTC-транзакций.